COMPREHENSIVE ENGINEERING REFERENCE

The SOG Bibl

The Complete Reference for the **Synthetic Occupancy Generator**

A deterministic synthetic benchmarking platform for entity resolution.

Author

Taha Mohammed

Version 2.0 | Engineering Complete | May 2026

SOG Synthetic Data Pipeline

Truth-preserving benchmark generation for realistic household and record change.

SOG

V2.0

Table of Contents

PART I: INTRODUCTION AND OVERVIEW

1. What is SOG
2. Why SOG Exists
3. The Complete Architecture

PART II: GETTING STARTED

4. Environment Setup
5. Phase 1 — Building the Baseline
6. Phase 2 — Running Your First Scenario

PART III: DEEP DIVE — HOW SOG WORKS

7. The Selection Engine
8. The Truth Simulator
9. The Emission Engine
10. Quality Reporting and Validation
11. Determinism and Data Model

PART IV: THE SCENARIOS

12. Complete Scenario Catalog
13. How to Choose and Use Scenarios

PART V: THE FRONTEND

14. The Streamlit Frontend

PART VI: TESTING AND QUALITY

15. Complete Test Suite
16. What Works and Known Issues

PART VII: EXTENDING SOG

17. How to Add Scenarios, Events, and Noise
18. Production Readiness

PART VIII: APPENDICES

A. File Map for New Engineers
B. Version 1 to Version 2 Comparison
C. Glossary

PART I

Introduction and Overview

What SOG is, why it exists, and how the system is organized

CHAPTER 1

What is SOG

SOG (Synthetic Occupancy Generator) is a deterministic synthetic data pipeline for entity-resolution benchmarking. It answers the practical question every ER system faces: *If my ER system is exposed to realistic household change, messy records, and partial coverage, how well does it recover the true entities?*

In its current form, SOG is a deterministic synthetic benchmarking platform built around one core idea: generate a believable underlying world, simulate how it changes over time, emit how different systems would observe it imperfectly, and preserve the answer key so it can be audited and scored exactly.

1.1 The Two-Phase Pipeline

Phase 1 generates a baseline population of synthetic people with demographics, addresses, names, SSNs, and phone numbers. This creates the starting world from which Phase 2 selects participants and begins simulation. Phase 1 is the starting world, not the benchmark outcome.

Phase 2 simulates life events (moves, cohabitation, births, divorces, roommate splits) on the selected population, then emits observed datasets with configurable noise and duplication. It produces:

- **Truth layer** (parquet): people, households, memberships, residence history, events
- **Observed layer** (CSV): one or more dataset files with realistic noise
- **Linking artifacts**: entity_record_map.csv, truth_crosswalk.csv, per-pair crosswalks
- **Metadata**: manifest.json, quality_report.json, scenario.yaml, scenario_selection_log.json

1.2 From Raw Data to Benchmark

SOG operates as a journey through six layered transformations:

- **Layer A — Prepared Inputs:** Seed assets (names, streets, cities, demographics, nicknames) are normalized into canonical lookup tables.
- **Layer B — Phase-1 Baseline:** Creates the starting synthetic population with controlled row redundancy.
- **Layer C — Scenario Selection:** Collapses Phase-1 rows into an entity view and attaches behavioral priors via stable hashing.
- **Layer D — Truth Simulation:** Changes the world through life events on normalized truth tables.
- **Layer E — Observed Emission:** Projects truth into imperfect observed datasets with noise, duplication, and partial coverage.
- **Layer F — Quality + Validation:** Measures the outcome and validates the artifact contract.

1.3 What Makes SOG Different

Table 1.1: SOG Differentiators

Property	What SOG Does	Why It Matters
Longitudinal identity	Models time-varying entities with households, memberships, residences, events	Real ER deals with changing identity, not isolated rows

Truth + observed separation	Produces both hidden truth tables and noisy observed CSVs	Precision/recall measured exactly against known ground truth
Benchmark topology	Single-dataset dedup, pairwise linkage, N-way linkage	One engine covers the full ER benchmarking spectrum
Explicit cardinality	one_to_one, one_to_many, many_to_one, many_to_many via record allocation	Difficulty is deliberate, not accidental
Two-level reproducibility	SHA256 hashing for traits + seeded RNG for simulation	Runs repeatable as codebase evolves
Full auditability	Run directory contains population, scenario, manifest, quality report, validator	Engineers inspect exactly how a benchmark was built

Quick Start

New users can run SOG in under 2 minutes:

```
python scripts/run_phase2_pipeline.py --scenario clean_baseline_linkage
```

CHAPTER 2

Why SOG Exists

Entity Resolution is the problem of determining which records across databases refer to the same real-world person. Every ER algorithm needs benchmark data with known ground truth. Real data cannot be shared due to privacy; hand-constructed datasets lack realistic difficulty. SOG fills this gap.

2.1 Five Messy Patterns of Real Data

Address Instability

People move. The same person appears at different addresses in different systems. SOG creates this through snapshot-based emission: Dataset A captures the start address, Dataset B captures the end address.

Household Dynamics

Couples form, children are born, divorces happen, young adults leave home. These events create shared addresses, name changes, and ambiguous linkage signals.

Field-Level Corruption

OCR errors, phonetic misspellings, nickname substitution, date transposition, SSN masking, missing fields. SOG applies 12 independently controllable noise types.

Coverage Asymmetry

System A covers 95% while System B covers 40%. An ER algorithm assuming every record has a match produces 60% false positives on the B side. SOG creates this through explicit overlap and coverage controls.

Duplication

The same person appears multiple times in the same file. SOG controls this through per-entity record counts scaled to configured targets.

2.2 The Gap in Existing Tools

Table 2.1: SOG vs. Alternative Approaches

Approach	Limitation	How SOG Differs
Faker / flat generators	Isolated records, no household/temporal structure	Household dynamics and life events
SDV / statistical	Requires real data access	Generates from public priors
Hand-curated (NCVR)	Small, static, limited variety	Arbitrarily large, 11+ scenarios
Real data with masking	Privacy risk, non-deterministic	Fully synthetic, deterministic, shareable

2.3 Determinism: The Foundation of Trust

SOG produces all outputs deterministically from a single seed. Given the same Phase-1 baseline, scenario YAML, and seed, outputs are identical. Determinism uses two levels:

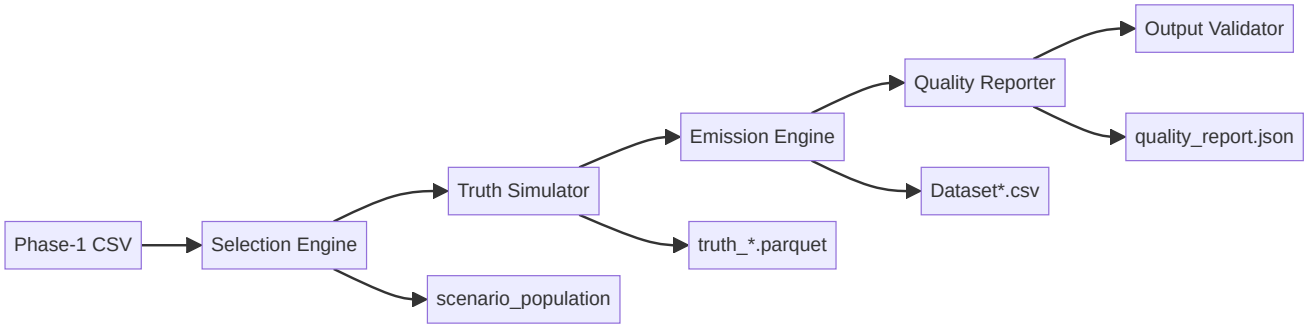
- **SHA256-based hashing** for entity-level decisions (selection, trait assignment) — stable regardless of population size or ordering
- **Seeded NumPy RNG** for simulation and emission — stable because modules iterate in sorted order

Adding person #10,001 to the Phase-1 file does not change the mobility score of person #3.

CHAPTER 3

The Complete Architecture

3.1 High-Level Pipeline



3.2 Source Modules

Table 3.1: Core Modules in src/sog_phase2/

Module	Purpose	Complexity
pipeline.py	Orchestrates full run: load YAML, select, simulate, emit, validate	Medium
simulator.py	Truth-layer state machine: households, events, residence	High
emission.py	Observed dataset generation: noise, duplication, coverage	High
selection.py	Population subsetting: entity view, latent traits, filtering	Medium
event_grammar.py	Event schema: MOVE, COHABIT, BIRTH, DIVORCE, LEAVE_HOME	Medium
constraints.py	Demographic constraints: marriage age, fertility, residence overlap	Medium
quality.py	Quality metrics: truth consistency, ER benchmark, drift	Medium
output_contract.py	Schema validation for all run artifacts	Medium
params.py	Census/CDC parameter loading	Low
scenario_catalog.py	Scenario registry (catalog.yaml)	Low

3.3 CLI Scripts

Table 3.2: CLI Entry Points

Script	Purpose
run_phase2_pipeline.py	CLI wrapper: --scenario single_movers runs full pipeline
generate_phase2_truth.py	Run truth simulation only
generate_phase2_observed.py	Run emission only
validate_phase2_outputs.py	Validate a completed run
build_phase2_params.py	Fetch Census/CDC data and build parameter tables
build_phase2_scenario_population.py	Rebuild scenario_population.parquet

3.4 Methodology

SOG uses a layered benchmark methodology: (1) Build a Phase-1 baseline, (2) Collapse to one entity row per PersonKey, (3) Assign deterministic latent propensities, (4) Simulate household and residence transitions on normalized truth tables, (5) Project truth into observed datasets at configured snapshots, (6) Distort with controlled duplication and field-level corruption, (7) Measure outcomes with quality metrics and validate against an explicit artifact contract.

The key choice is the separation between truth creation and observed emission. Truth tables stay internally consistent and auditable. Observed files carry the ambiguity, incompleteness, and corruption that ER systems must solve.

3.5 Core Algorithmic Ideas

Table 3.3: Algorithmic Choices and Their Rationale

Problem	Choice	Why It Matters

Stable per-person variation	SHA256 deterministic values keyed by (seed, salt, PersonKey)	Adding/reordering rows does not reshuffle traits
Realistic event frequency	Convert annual rates to step probabilities with compounding	Monthly simulation approximates annual targets
Prevent impossible same-step behavior	Ordered event phases plus per-step locking	No divorce+cohabit+move in same step
Achieve requested overlap	Overlap-first allocation, then side-specific sampling	Overlap mathematically consistent with coverage
Reusable answer keys	Canonical entity_record_map.csv plus pairwise projections	Same run supports multiple evaluation views

3.6 Repository Structure

```
SOG/
|-- src/sog_phase1/           # Phase 1: baseline generation
|   |-- preprocess.py         # Source table preparation
|   |-- config.py             # YAML parsing
|   |-- generator.py          # Baseline synthesis
|   |-- nicknames.py          # Name variant catalog
|   |-- redundancy.py         # Record allocation
|-- src/sog_phase2/           # Phase 2: core library
|   |-- pipeline.py           # Orchestrator
|   |-- simulator.py          # Truth state machine
|   |-- emission.py           # Observed dataset + noise
|   |-- selection.py          # Population subsetting
|   |-- event_grammar.py      # Event definitions
|   |-- constraints.py        # Demographic rules
|   |-- quality.py            # Quality metrics
|   |-- output_contract.py    # Artifact validator
|   |-- params.py             # Census/CDC parameter loading
|   |-- scenario_catalog.py   # Scenario registry
|-- scripts/                  # CLI entry points
|-- tests/                    # 322 tests (318 + 4 frontend)
|-- phase1/                   # Phase 1 inputs/outputs
|-- phase2/                   # Scenario configs and run outputs
|-- Data/phase2_params/       # Demographic priors
|-- frontend/                 # Streamlit UI
|-- configs/                  # Configuration files
```

Reading Order for New Engineers

1. This document → 2. pipeline.py → 3. A scenario YAML → 4. simulator.py → 5. emission.py

PART II

Getting Started

From first clone to first validated scenario run

CHAPTER 4

Environment Setup

4.1 Prerequisites

Table 4.1: Required Software

Software	Version	Purpose
Python	3.10+	Runtime for all SOG scripts
Git	Any recent	Repository clone and versioning
Terminal	PowerShell, CMD, or bash	Command execution

```
python --version   # Should show 3.10+
git --version
```

4.2 Clone and Setup

```
git clone https://github.com/MaTaha-ualr/Synthetic-Occupancy-Generator-data_pipeline.git
cd Synthetic-Occupancy-Generator-data_pipeline
python -m venv .venv
# Windows: .\.venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
pip install -r requirements-dev.txt
```

4.3 Build the Prepared Cache

```
python scripts/build_prepared.py --raw-root . --prepared-dir prepared
```

Expected outputs: first_names.parquet, last_names.parquet, streets.parquet, cities.parquet, states.parquet, demographics.json, nicknames.json, prepared_manifest.json.

4.4 Build Phase-2 Parameters

```
python scripts/build_phase2_params.py
```

Priors are shipped pre-built in Data/phase2_params/. This step only needed to refresh with newer published rates. It hits the Census API.

4.5 Run the Test Suite

```
python -m pytest -q
```

Expected: **322 passed** (318 core + 4 frontend). If failures occur, check that prepared cache and Phase-2 parameters are built.

4.6 Recreating a Clean Working State

```
python scripts/build_prepared.py
python scripts/generate_phase1.py --config configs/phase1.yaml --prepared-dir prepared --overwrite
# Copy canonical baseline
cp phase1/outputs/Phase1_people_addresses.csv phase1/outputs_phase1/
cp phase1/outputs/Phase1_people_addresses.manifest.json phase1/outputs_phase1/
cp phase1/outputs/Phase1_people_addresses.quality_report.json phase1/outputs_phase1/
python scripts/build_phase2_params.py
python -m pytest -q
```

Git Ignore These Directories

Do not commit: phase1/prepared/, phase1/outputs/, phase2/runs/, phase2/.sog_*/. However, phase1/outputs_phase1/README.md and phase2/runs/.gitkeep are intentionally versioned.

CHAPTER 5

Phase 1 — Building the Baseline

Phase 1 creates the synthetic person-and-address population that Phase 2 uses as its starting world. Understanding Phase 1 is essential because Phase 2 consumes an entity-centric view of this baseline, not the raw rows directly.

5.1 What Phase 1 Produces

Output: `phase1/outputs/Phase1_people_addresses.csv` — each row is a person-record. A single person can have multiple rows (redundancy).

Key columns: RecordKey, PersonKey, EntityRecordIndex, FormalFirstName, FirstName, MiddleName, LastName, Suffix, FullName, Gender, Ethnicity, DOB, Age, AgeBin, SSN, Phone, ResidenceAddress, ResidenceCity, ResidenceState, ResidencePostalCode, MailingAddress, MailingCity, MailingState, MailingPostalCode, MailingAddressMode.

RecordKey is unique per row. PersonKey identifies the entity and can repeat when redundancy is enabled. Postal codes are 5-digit strings.

5.2 Running Phase 1

```
python scripts/generate_phase1.py --config configs/phase1.yaml --prepared-dir prepared --overwrite
```

Produces three files: the CSV, a manifest.json, and a quality_report.json.

5.3 Key Configuration Options

Table 5.1: Phase-1 Configuration

Setting	What It Controls

phase1.n_entities	Number of unique people
phase1.n_records	Total output row count
phase1.redundancy.enabled	False: n_records == n_entities. True: n_records > n_entities
phase1.redundancy.shape	balanced or heavy_tail distribution
phase1.nicknames.enabled	Nickname variation on/off
phase1.nicknames.usage_pct	Target % of rows using nicknames
phase1.seed	Random seed for reproducibility
phase1.age_bins	Active age ranges and percentages
phase1.distributions.gender	Gender target percentages
phase1.address	House/apartment mix and mailing behavior

5.4 Safe First Edits

- Change n_entities and n_records together for new runs
- Keep the same seed for reproducible reruns
- Keep age_bins.auto_normalize: true unless you want strict sum checks
- For pair duplicates: set collision_group_min_size: 2 and collision_group_max_size: 2

5.5 Troubleshooting

Table 5.2: Phase-1 Common Issues

Error	Cause	Fix
"Prepared cache incomplete"	Cache not built	Re-run build_prepared.py
"Output already exists"	Previous run without overwrite	Add --overwrite
Strange percentages	Misconfigured age bins	Check enabled_bins_only, auto_normalize
Import/module errors	Wrong venv	Confirm virtual environment is active

CHAPTER 6

Phase 2 — Running Your First Scenario

6.1 One-Command Method

```
python scripts/run_phase2_pipeline.py --scenario single_movers
```

This performs selection, simulation, emission, quality reporting, and validation. Output lands in `phase2/runs/YYYY-MM-DD_single_movers_seedN/`.

6.2 Three-Step Method

```
# Step 1: Truth Generation
python scripts/generate_phase2_truth.py --run-id 2026-03-14_roommates_split_seed20260314
# Step 2: Observed Emission
python scripts/generate_phase2_observed.py --run-id 2026-03-14_roommates_split_seed20260314
# Step 3: Validation
python scripts/validate_phase2_outputs.py --run-id 2026-03-14_roommates_split_seed20260314
```

Run-id format: `YYYY-MM-DD_<scenario_id>_seed<seed>`

6.3 Run Directory Contents

```
phase2/runs/2026-03-10_single_movers_seed20260310/
|-- truth_people.parquet
|-- truth_households.parquet
|-- truth_household_memberships.parquet
|-- truth_residence_history.parquet
|-- truth_events.parquet
|-- scenario_population.parquet
|-- DatasetA.csv
|-- DatasetB.csv
|-- entity_record_map.csv
|-- truth_crosswalk.csv
|-- manifest.json
|-- quality_report.json
|-- scenario.yaml
|-- scenario_selection_log.json
```

6.4 Inspecting Core Artifacts

After every run, review in order:

1. **truth_events.parquet** — Did the primary events occur?
2. **entity_record_map.csv** — Does topology match intent?
3. **truth_crosswalk.csv** — Does linking match cardinality?
4. **quality_report.json** — Did metrics land in expected ranges?

6.5 Evaluating Your ER System

Pairwise: Run ER on DatasetA.csv + DatasetB.csv, compare predictions with truth_crosswalk.csv.

Single-dataset dedup: Use entity_record_map.csv as truth for record-to-person grouping.

N-way: Compare predictions against per-pair truth crosswalks in manifest.json.

Track: Precision, Recall, F1, False Positives, False Negatives.

6.6 Complete End-to-End

```
# 1. Setup
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && pip install -r requirements-dev.txt
# 2. Build cache and baseline
python scripts/build_prepared.py
python scripts/generate_phase1.py --config configs/phase1.yaml --prepared-dir prepared --
overwrite
cp phase1/outputs/Phase1_people_addresses.* phase1/outputs_phase1/
python scripts/build_phase2_params.py
# 3. Run scenario
python scripts/run_phase2_pipeline.py --scenario single_movers
# 4. Validate
python -m pytest -q
```

PART III

Deep Dive — How SOG Works

Pipeline internals, algorithms, and design decisions

CHAPTER 7

The Selection Engine

The selection engine takes the full Phase-1 population and produces a scenario-specific cohort with assigned latent propensity traits. It transforms the data model from record-centric to entity-centric.

7.1 The Four Steps

Step A — Entity View

Collapse multi-record people into one row: Representative = first record (by EntityRecordIndex, RecordKey); ResidenceType = last record's; RecordsCount = number of rows; Redundancy = "multi_record" if RecordsCount > 1.

Output: DataFrame[PersonKey, AgeBin, Gender, Ethnicity, ResidenceType, RedundancyProfile]

Step B — Latent Traits

Assign propensity scores using a two-component formula:

```
mobility_score    = clip(mobility_base    + (hash_unit - 0.5) * 0.08, 0, 1)
partnership_score = clip(partnership_base + (hash_unit - 0.5) * 0.30, 0, 1)
fertility_score   = clip(fertility_base   + (hash_unit - 0.5) * 0.30, 0, 1)
```

mobility_base comes from Census ACS data for the age cohort. hash_unit = SHA256(seed|"mobility"|PersonKey) converted to float in [0,1]. Jitter bounds: mobility +/- 0.04, partnership/fertility +/- 0.15.

Scores are bucketed: low, medium, high using scenario-configurable thresholds:

```
mobility_bucket = "low" if score <= 0.09; "high" if score >= 0.18; "medium" otherwise
```

Step C — Filter

Apply optional demographic filters: age_bins, genders, ethnicities, residence_types, redundancy_profiles, mobility_buckets. Empty list means no filtering.

Step D — Sample

Deterministically select N people: mode "all" takes everyone; "count" takes exactly N via seeded RNG; "pct" takes round(pct/100 * len).

7.2 Why SHA256 for Jitter

numpy.random depends on call order, making it fragile when populations change. SHA256 produces a stable float for each (seed, salt, PersonKey) triple:

```
float_value = SHA256(f"{seed}|{salt}|{person_key}")
            -> first 8 bytes -> uint64 -> divide by 2^64-1
```

This preserves reproducibility at the entity level (same seed + PersonKey = same score) and heterogeneity at the population level (different PersonKeys = different scores).

7.3 Audit Trail

scenario_selection_log.json stores filter counts per stage, selected entity count, and a checksum over the ordered PersonKey list. Selection is fully auditable.

CHAPTER 8

The Truth Simulator

The simulator is a discrete-time state machine advancing through monthly steps, rolling dice to decide life events. It is the core of SOG's longitudinal identity modeling.

8.1 State Machine Initialization

For each selected person, the simulator creates: a solo household (HH_SIM_XXXXXX), an address from Phase-1, a HEAD membership, and a residence record. If roommate_group_share_pct > 0, eligible people are pre-grouped into shared baseline households.

8.2 Rate-to-Probability Conversion

```
p_step = 1 - (1 - p_annual)^(1/steps_per_year)
```

Example: 12% annual move rate becomes ~1.06% monthly. Running 12 monthly steps approximates the annual rate, not 12x it.

8.3 Event Processing Order

Table 8.1: Event Processing Order

Order	Event	Rationale
1	DIVORCE	Break couples before new couples form
2	COHABIT	Pair up newly single people
3	BIRTH	New couples can reproduce immediately
4	LEAVE_HOME	Young adults leave shared households
5	MOVE	Moves don't interfere with household formation

Per-step locking prevents impossible same-step behavior.

8.4 Probability Formulas

Table 8.2: Event Probability Formulas

Event	Formula
DIVORCE	divorce_base * (0.7 + (1.0 - avg_partnership))

COHABIT	cohabit_base * (0.5 + partnership)
BIRTH	birth_base * (0.4 + fertility)
MOVE	move_base * (0.5 + mobility)

These simple, interpretable formulas let users reason about scenario outcomes without reverse-engineering a black box.

8.5 Interval-Based State Management

SOG is interval-based, not overwrite-based. A move closes the current residence interval and opens a new one. _SimulationState keeps: static attributes, current pointers (household, address), open-interval indexes, active_couples, and monotonic counters for all key types.

8.6 Constraint Enforcement

Constraints checked before each event: min_marriage_age, max_partner_age_gap, fertility_age_range, household_size_max. Checked before, not after.

8.7 Truth Tables Output

- **truth_people:** All people including children born during simulation
- **truth_households:** All households with type and lifespan
- **truth_memberships:** Person-to-household assignments with intervals
- **truth_residence:** Person-to-address assignments with intervals
- **truth_events:** All life events with type, date, and event fields

CHAPTER 9

The Emission Engine

The emission engine takes the truth layer and produces observed datasets with noise, duplication, and controlled coverage.

9.1 The Five Steps

Step A — Entity Appearance

Single-dataset: sample by appearance_pct. Pairwise: compute overlap, then A-only, B-only, add late-born to B. Multi-dataset: each dataset gets its own entity set with controlled pairwise overlap. The overlap-first algorithm ensures requested overlap remains consistent with coverage targets.

Step B — Record Count Allocation

Start with 1 record per entity. Force at least 2 if match mode requires multi-record. Add random extras to hit duplication targets.

Table 9.1: Cardinality Requirements

Mode	A-side	B-side
one_to_one	1	1
one_to_many	1	≥2
many_to_one	≥2	1
many_to_many	≥2	≥2

Step C — Snapshot Building

Resolve each person's active address on the snapshot date, recompute age, attach address details, stamp SourceSnapshotDate. Dataset A typically snapshots at simulation_start; Dataset B at simulation_end.

Step D — Noise Application

Table 9.2: 12 Noise Types

Type	Effect	Example
name_typo	Replace random character	"Smith" → "Smoth"
dob_shift	Shift -3 to +3 days	"1985-06-15" → "1985-06-17"
ssn_mask	Mask to ***-**-last4	"123-45-6789" → "***-**-6789"
phone_mask	Clear phone	"555-1234" → ""
middle_name_missing	Drop middle name	"John Q. Public" → "John Public"
phonetic_error	Substitute phonetic cluster	"ph" → "f"
ocr_error	Substitute OCR-confused char	"O" → "0"
nickname	Replace with nickname	"William" → "Bill"
date_swap	Swap month and day	"06/15" → "15/06"
zip_digit_error	Flip one ZIP digit	"90210" → "90211"
suffix_missing	Drop suffix	"Jr." removed
address_missing	Clear address fields	All address fields blank

Step E — Linking Artifacts

entity_record_map.csv: [PersonKey, DatasetId, RecordKey] for every emitted record. The canonical truth map.

truth_crosswalk.csv: For pairwise mode, direct A-to-B linking by PersonKey. In many_to_many mode, emits the Cartesian product between A and B records per person.

CHAPTER 10

Quality Reporting and Validation

10.1 Quality Reporter

Computes three categories of metrics:

Truth Consistency: Event age validation, time overlap errors, household size constraints, couple colocation.

Scenario Metrics: Event counts (MOVE, COHABIT, BIRTH, DIVORCE, LEAVE_HOME), moves-per-person, household type shares.

ER Benchmark Metrics: Cross-file overlap, within-file duplicate rates, attribute drift rates (name, address, phone), match cardinality achieved, crosswalk ambiguity.

10.2 Output Contract Validator

Verifies: required files exist, column schemas present, run identity consistent across folder/manifest/YAML, event grammar valid, truth constraints valid, topology-specific requirements met (pairwise vs N-way crosswalks).

10.3 Reading the Quality Report

Key paths to inspect:

- `scenario_metrics.event_counts` — Did primary events occur?
- `er_benchmark_metrics.cross_file_overlap` — Is coverage expected?
- `er_benchmark_metrics.match_cardinality_achieved` — Correct cardinality?
- `er_benchmark_metrics.within_file_duplicate_rates` — Duplication at target?
- `er_benchmark_metrics.attribute_drift_rates` — Noise visible?
- `truth_consistency` — Any constraint violations?

CHAPTER 11

Determinism and Data Model

11.1 The Determinism Model

Strategy 1 — SHA256 Hashing: For entity-level decisions. Produces a stable float for each (seed, salt, PersonKey) triple.

```
float_value = SHA256(f"{seed}|{salt}|{person_key}") -> first 8 bytes -> uint64 -> / (2^64-1)
```

Strategy 2 — Seeded NumPy RNG: For simulation and emission. Both modules iterate in stable sorted order. Derived seed for roommate grouping: seed + 1879.

11.2 Data Model

Truth is normalized; observed is denormalized.

Table 11.1: Truth Layer Schema

Table	Primary Key	Key Columns

truth_people	PersonKey	Name, Gender, Ethnicity, DOB, Age, SSN, Phone
truth_households	HouseholdKey	Type, StartDate, EndDate
truth_memberships	PersonKey+HHKey+Start	Role, EndDate
truth_residence	PersonKey+AddrKey+Start	EndDate
truth_events	EventKey	Type, Date, event fields

entity_record_map.csv is the authoritative bridge: PersonKey (FK), DatasetId, RecordKey.

truth_crosswalk.csv (pairwise): PersonKey, A_RecordKey, B_RecordKey.

PART IV

The Scenarios

Complete catalog of 11 canonical benchmarking scenarios

CHAPTER 12

Complete Scenario Catalog

12.1 Overview

Table 12.1: All 11 Canonical Scenarios

Scenario	Difficulty	Primary Event	Match Mode	ER Stress
clean_baseline_linkage	Beginner	MOVE (low)	one_to_one	Sanity check
single_movers	Beginner	MOVE	one_to_one	Address change
couple_merge	Intermediate	COHABIT	one_to_many	Household formation
family_birth	Intermediate	BIRTH	many_to_one	Household growth
roommates_split	Intermediate	LEAVE_HOME+MOVE	one_to_many	High churn
high_noise_identity_drift	Intermediate	MOVE	one_to_one	Field corruption
low_overlap_sparse_coverage	Intermediate	MOVE	one_to_one	Weak coverage
asymmetric_source_coverage	Intermediate	MOVE	one_to_one	Unequal sources
high_duplication_dedup	Advanced	MOVE (bg)	single_dataset	Within-file dedup
divorce_custody	Advanced	DIVORCE+COHABIT	many_to_many	Family ambiguity
three_source_partial_overlap	Advanced	MOVE	N-way (3)	Multi-source

Start Here

New to SOG? Run in order: 1. clean_baseline_linkage (sanity), 2. single_movers (simplest real), 3. couple_merge (households).

12.2 Quick Decision Guide

Table 12.2: Choose Your Scenario

Goal	Scenario
Sanity check	clean_baseline_linkage
Address-change benchmarking	single_movers
Household formation	couple_merge
Child-creation, household expansion	family_birth
Worst family ambiguity	divorce_custody
High churn, roommates	roommates_split
Heavy field corruption	high_noise_identity_drift
Weak coverage, many non-matches	low_overlap_sparse_coverage

Unequal source coverage	asymmetric_source_coverage
Single-file dedup	high_duplication_dedup
Three-source linkage	three_source_partial_overlap

12.3 Scenario Details

clean_baseline_linkage

Two systems describing mostly the same people with very little drift, duplication, or ambiguity. Low-noise, high-overlap pairwise. Use as a sanity check. Inspect truth_crosswalk.csv for one-to-one pairs; quality_report.json for low noise and high overlap.

single_movers

People relocate between snapshots. Address-change benchmarking. one_to_one matching. ER stress: matching the same person when address is unstable. Inspect truth_events for MOVE; entity_record_map for simple linking.

couple_merge

Two people form a household after cohabitation. one_to_many with B as messier side. ER stress: distinguishing true shared-address couples from coincidental co-residence. Inspect truth_events for COHABIT; truth_household_memberships for shared households.

family_birth

Households grow as children are added. many_to_one. ER stress: linking expanded households; multiple A-side records collapsing to one B-side. Inspect truth_events for BIRTH; truth_people for child keys.

divorce_custody

Households form then split. Most structurally ambiguous. many_to_many with elevated duplication on both sides. ER stress: separating prior ties from current structure; avoiding false positives from shared history. Inspect truth_events for DIVORCE; truth_households for post_divorce types.

roommates_split

Young adults start in roommate groups then split. one_to_many with high B-side duplication. ER stress: separating roommates from true entity duplication. Inspect truth_events for LEAVE_HOME; truth_household_memberships for larger shared households.

high_noise_identity_drift

Heavy corruption: OCR, nicknames, phonetic, suffix loss, date formatting. one_to_one with much higher B noise. ER stress: recovering matches under realistic field corruption. Inspect attribute_drift_rates in quality_report.

high_duplication_dedup

One system with many duplicate records per person. Single dataset, no crosswalk. ER stress: clustering duplicate records under one entity. Inspect entity_record_map.csv for repeated RecordKey per PersonKey.

low_overlap_sparse_coverage

Two systems each covering only part of the population. Low overlap. ER stress: maintaining precision when true matches are rare. Inspect truth_crosswalk.csv for few linked entities.

asymmetric_source_coverage

One broad source, one narrow. Many unmatched A-only entities. ER stress: interpreting recall when sources are imbalanced. Inspect entity_record_map.csv for per-dataset entity count disparity.

three_source_partial_overlap

Three systems with overlapping but not identical coverage. N-way with per-pair crosswalks. ER stress: reconciling identities across more than two systems. Inspect manifest.json for per-pair crosswalk inventory.

12.4 Future Extensions

Table 12.3: Planned Future Scenarios

Scenario Family	Required Engine Work
name_change_lifecycle	Activate NAME_CHANGE event
death_survivor_persistence	Activate DEATH event
adoption_blended_family	Activate ADOPTION event

CHAPTER 13

How to Choose and Use Scenarios

13.1 Scenario YAML Structure

Table 13.1: YAML Top-Level Sections

Section	Purpose
scenario_id	Unique identifier
seed	Deterministic random seed
phase1	Path to baseline data and manifest
parameters	Event rates (move, cohabit, birth, divorce)
simulation	Granularity, start_date, periods
emission	Match mode, datasets, overlap, duplication, noise
selection	Filters, sample mode, propensity thresholds
constraints	Marriage age, fertility range, partner gap
quality	Household size range

13.2 Emission Configuration

crossfile_match_mode: single_dataset | one_to_one | one_to_many | many_to_one | many_to_many. **datasets:** Each has dataset_id, filename, snapshot (simulation_start/end), appearance_pct, duplication_pct, noise.* percentages.

13.3 Failure Diagnosis

- **No primary event:** Check truth_events.parquet, event_counts in quality_report
- **Wrong overlap:** Check entity_record_map.csv, cross_file_overlap in quality_report
- **Wrong cardinality:** Check match_cardinality_achieved in quality_report
- **Noise not visible:** Check attribute_drift_rates in quality_report
- **Wrong households:** Check truth_households, truth_memberships, truth_consistency

PART V

The Frontend

Streamlit UI for scenario drafting, execution, and results

CHAPTER 14

The Streamlit Frontend

SOG includes a Streamlit UI that wraps the underlying Python tools into an interactive web application. It provides scenario drafting, execution orchestration, result visualization, and export packaging.

14.1 Capabilities

Table 14.1: Frontend Features

Capability	Description
Scenario Drafting	Edit and save scenario YAML files through the UI
Phase-2 Orchestration	Run scenarios and manage run directories
Result Summaries	Display run metadata, quality metrics, event counts
Chart Rendering	Visualize household types, event distributions, drift
Artifact Download	Package and export run artifacts
Export Packaging	Bundle truth + observed + metadata for sharing

14.2 Architecture

Streamlit App (chatbot_production.py) → sog_tools.py (backend functions) → Phase-2 Pipeline. Transient state written to phase2/.sog_jobs/, .sog_sessions/, .sog_charts/, .sog_exports/.

14.3 Launching

```
.\run_frontend.ps1
```

Activates venv and launches Streamlit. Default URL: http://localhost:8501

14.4 Configuration

Requires Anthropic API key for AI-assisted scenario editing:

```
$env:ANTHROPIC_API_KEY = "your_key"
```

14.5 Tests

4 frontend tests: scenario inference, run orchestration, job lifecycle, export packaging.

```
python -m pytest tests/frontend/ -v
```

14.6 Troubleshooting

Table 14.2: Frontend Issues

Issue	Solution
Frontend fails to start	Check venv active; reinstall requirements
Edits not saving	Check ANTHROPIC_API_KEY; check file permissions
Charts not rendering	Delete phase2/.sog_charts/ to clear cache
Run not in history	Check phase2/.sog_jobs/ exists and is writable

CHAPTER 14 ADDENDUM

Hosted model providers and export formats

Effective May 5, 2026. This page supersedes the Anthropic-only configuration wording on pages 28-29.

Quality-first assistant policy

Every assistant role now uses the same selected top-tier hosted model. There is no fast/basic classifier, cheap router, local transformers serving path, local model download, or local GPU requirement.

Approved hosted providers

Together AI:	SOG_LLM_PROVIDER=together; SOG_LLM_MODEL=zai-org/GLM-5.1; TOGETHER_API_KEY
NVIDIA NIM:	SOG_LLM_PROVIDER=nvidia; SOG_LLM_MODEL=moonshotai/kimi-k2.6; NVIDIA_API_KEY
Anthropic:	SOG_LLM_PROVIDER=anthropic; SOG_LLM_MODEL=claude-opus-4-7; ANTHROPIC_API_KEY

NVIDIA/Kimi runtime notes

- Hosted API endpoint: https://integrate.api.nvidia.com/v1. Kimi responses are streamed internally.
- The adapter disables Kimi thinking output, uses temperature 0.0, and defaults to a 180-second timeout.
- Keep moonshot.txt local only. Put NVIDIA_API_KEY in .env or paste it into the UI access gate.
- Developer/free hosted endpoints are for development and testing; review NVIDIA terms before production use.

Export formats

The artifact shelf can download individual tabular artifacts or a full-run zip in:
csv, tsv, txt, xlsx, json, jsonl, parquet.
Conversions are additive: canonical run outputs under phase2/runs/ are not rewritten.
CLI: python scripts/export_run.py --run-id <run_id> --format xlsx

Guardrails and validation

The app blocks prompt-injection requests for secrets or hidden instructions, redacts recognizable secrets, restricts model tool calls to declared SOG tools, validates output paths stay inside the repo, and caps agent iterations. After the NVIDIA/Kimi provider update, local validation passed: 333 pytest tests.

See docs/FRONTEND_RUNBOOK.md and frontend/README.md for current setup details.

PART VI

Testing and Quality

The complete test suite, evidence, and known issues

CHAPTER 15

Complete Test Suite

SOG has 322 tests (318 core + 4 frontend), all passing.

15.1 Test Architecture

Table 15.1: Test Categories

Category	Tests	Time	Proves
Unit / Quick	~260	~10s	Component behavior in isolation
Integration / Extended	~20	~20s	End-to-end pipeline
Regression / Smoke	~28	~45s	All 11 scenarios execute
Frontend	4	~5s	UI orchestration
Total	**322**	**~80s**	**Comprehensive coverage**

15.2 Coverage by Module

Table 15.2: Test Files

File	Module	Coverage
test_selection.py / _extended.py	selection.py	Strong
test_simulator.py / _extended.py	simulator.py	Strong
test_emission.py / _extended.py	emission.py	Strong
test_quality.py	quality.py	Strong
test_output_contract.py	output_contract.py	Strong
test_constraints.py	constraints.py	Good
test_pipeline_smoke.py	pipeline.py	Good
test_scenario_smoke.py	Scenario YAMLs	Good
test_phase1_pipeline.py	Phase 1	Strong
frontend/ tests	frontend/	Basic

15.3 What the Tests Prove

Determinism: Same seed + same YAML = same output. Verified for selection, emission, and simulation.

Structural Correctness: Event ordering constraints enforced (no cohabit before age 16, no births outside fertility range, no overlapping residence intervals).

Coverage and Cardinality: Every scenario produces the requested match mode. Smoke tests verify all 11 scenarios.

Noise Accounting: Noise counts from observed data match YAML targets. Tracked per-type and verified.

15.4 Running Tests

```
python -m pytest -q                     # All 322 tests, ~80s
python -m pytest -q -m "not extended and not regression"  # Unit only, ~10s
python -m pytest -q -m extended         # Integration, ~20s
python -m pytest -q -m regression       # Regression, ~45s
python -m pytest tests/frontend/ -v     # Frontend, ~5s
```

15.5 Test Gaps

- No performance benchmarks at 100K+ scale
- No property-based (Hypothesis) testing
- No chaos/fuzz testing
- Frontend has 4 orchestration tests but no component unit tests

CHAPTER 16

What Works and Known Issues

16.1 What Works Well

Determinism: SHA256 + seeded RNG produces stable outputs. Verified by unit tests.

Scenario System: 11 scenarios run end-to-end. All produce truth and observed matching requested topology.

Noise Precision: Rates land within ~0.5% of YAML targets.

Schema Evolution: Frozen dataclasses protect pipeline from schema changes.

Validation: Every run produces manifest, quality report, and validator result.

16.2 MUST FIX Issues

Table 16.1: MUST FIX

#	Issue	Risk
1	Count discrepancy between quality report and manifest	Users see inconsistent numbers
2	No atomic writes during generation	Partial runs leave invalid dirs
3	Silent error swallowing in simulator	Failures produce bad data unnoticed
4	Windows absolute paths in run names	Breaks cross-platform use

16.3 SHOULD FIX Issues

Table 16.2: SHOULD FIX

#	Issue	Risk

5	Frontend silent error swallowing	Users don't know if edits saved
6	CSV string formatting fragile	Postal codes may go numeric; SSNs lose leading zeros
7	Monolithic simulator (~1000+ lines)	Hard to maintain and extend
8	No frontend component unit tests	UI regressions not caught by CI

16.4 Known Limitations

Table 16.3: Known Limitations

Limitation	Impact
Python 3.12 required on macOS (psycopg2-binary)	Non-macOS users adjust deps
Dependencies not pinned	Future numpy/pandas may break determinism
Roommate grouping uses derived seed (seed + 1879)	Documented edge case
O(n*m) quality computation for some checks	100K+ populations slower reporting

16.5 Readiness Assessment

Table 16.4: Readiness by Dimension

Dimension	Score	Evidence
Correctness	9/10	322 tests pass, all scenarios validate
Reliability	6/10	No atomic writes, silent errors possible
Portability	5/10	Windows paths, unpinned deps
Documentation	8/10	This bible + inline docs + 9 companion docs
Maintainability	6/10	Monolithic simulator, no frontend component tests
Overall	**7/10**	**Ready for research; path to production clear**

16.6 Who Can Use SOG Now

Table 16.5: User Readiness

User	Ready?	Caveat
Local researcher	Yes	None
CI/CD pipeline	With caution	Fix Issue 2 first
Web deployment	No	Fix Issues 1-4 first
Cross-platform team	With caution	Fix Issue 4 first

PART VII

Extending SOG

How to add scenarios, events, noise, and understand readiness

CHAPTER 17

How to Add Scenarios, Events, and Noise

17.1 Adding a New Scenario

Step 1: Copy an existing YAML in phase2/scenarios/ as template. Modify scenario_id, seed, parameters, simulation, emission, selection.

Step 2: Register in phase2/scenarios/catalog.yaml.

Step 3: Run: `python scripts/run_phase2_pipeline.py --scenario new_scenario`

Step 4: Inspect truth_events.parquet and quality_report.json.

Step 5: Add a smoke test in test_pipeline_smoke.py. The framework auto-registers every scenario in catalog.yaml.

17.2 Adding a New Event Type

Step 1: Register in event_grammar.py with required fields.

Step 2: Implement in simulator.py. Place in correct order (DIVORCE → COHABIT → BIRTH → LEAVE_HOME → MOVE → YOUR_EVENT).

Step 3: Add demographic rules in constraints.py.

Step 4: Add unit test in test_simulator.py and smoke test in test_pipeline_smoke.py.

17.3 Adding a New Noise Type

Step 1: Implement in emission.py. Must be deterministic, track count in noise_counts, apply to single field.

Step 2: Register percentage in scenario YAML under emission.datasets.*.noise.

Step 3: Add to quality.py schema for counting and reporting.

Step 4: Add unit test asserting: noise_count > 0, transformed != original, quality report matches.

17.4 Common Pitfalls

Table 17.1: Pitfalls

Pitfall	Solution
Forgetting to add scenario to catalog.yaml	Add immediately; tests catch it
Forgetting constraints.py for new events	Gate new event eligibility
Not tracking noise in noise_counts	Quality report shows zero
Wrong event ordering	Respect phase ordering
Changing YAML without updating tests	Smoke tests catch mismatches

17.5 Validation

```
python -m pytest -q   # 322 must all pass
python scripts/run_phase2_pipeline.py --scenario your_scenario
python scripts/validate_phase2_outputs.py --run-id ...
```

CHAPTER 18

Production Readiness

18.1 Current State

SOG V2.0 is a complete, working ER benchmarking platform. 322 tests pass. 11 scenarios ship and validate. All outputs are deterministic and reproducible. Every run passes the output contract validator. Zero constraint violations in regression. Noise rates within ~0.5% of targets. Full documentation ecosystem exists.

18.2 What Earns 7/10

- Every shipped scenario is end-to-end validated
- Determinism is verifiable and unit-tested
- Schema evolution protected by frozen dataclasses
- Output contract catches structural bugs
- Comprehensive documentation for all use cases

18.3 What Prevents Higher Score

- 4 MUST FIX issues (count discrepancy, atomic writes, silent errors, Windows paths)
- 4 SHOULD FIX issues (frontend transparency, CSV formatting, monolithic simulator, frontend tests)
- Python 3.12 requirement on macOS
- Unpinned dependencies

18.4 Bottom Line

SOG is ready for research, local benchmarking, and controlled CI use once Issues 1-4 are addressed. Not yet ready for public web deployment without portability fixes. The documentation is complete. The code is well-structured. The test suite is comprehensive. The path from 7/10 to 9/10 is clear and bounded.

PART VIII

Appendices

Reference material for engineers and users

APPENDIX A

File Map for New Engineers

A.1 Repository Structure

```
SOG/
|-- src/sog_phase1/           # Phase 1 baseline generation
|   |-- preprocess.py         # Source table preparation
|   |-- config.py             # YAML parsing
|   |-- generator.py          # Baseline synthesis
|   |-- nicknames.py          # Name variant catalog
|   |-- redundancy.py         # Record allocation policy
|-- src/sog_phase2/           # Phase 2 core library
|   |-- pipeline.py           # Orchestrator
|   |-- simulator.py          # Truth state machine
|   |-- emission.py           # Observed dataset + noise
|   |-- selection.py          # Population subsetting
|   |-- event_grammar.py      # Event definitions
|   |-- constraints.py        # Demographic rules
|   |-- quality.py            # Quality metrics
|   |-- output_contract.py    # Artifact validator
|   |-- params.py             # Census/CDC parameter loading
|   |-- scenario_catalog.py   # Scenario registry
|-- scripts/                  # CLI entry points
|-- tests/                    # 322 tests (318 + 4 frontend)
|-- phase1/                   # Phase 1 inputs/outputs
|-- phase2/                   # Scenario configs and run outputs
|-- Data/phase2_params/       # Demographic priors
|-- frontend/                 # Streamlit UI
|-- configs/                  # Configuration files
```

A.2 Where to Find Things

Table A.1: Quick Reference

Looking for...	Go to...
Run a scenario	scripts/run_phase2_pipeline.py
How noise works	src/sog_phase2/emission.py
Add a scenario	Chapter 17 of this document
Test examples	tests/test_phase2_*_extended.py
Frontend code	frontend/sog_tools.py
Event definitions	src/sog_phase2/event_grammar.py
Quality metrics	src/sog_phase2/quality.py
Validation rules	src/sog_phase2/output_contract.py

APPENDIX B

Version 1 to Version 2 Comparison

B.1 SOG Version 1

V1 was an innovative but architecturally incomplete synthetic data system. Core architecture: seed-and-row generator selecting from a seeded population, creating cross-table references, populating schema fields, outputting CSVs. It had a working output validator but significant limitations: monolithic architecture, completely random generation, no household dynamics, constant static identity, no longitudinal truth layer, flat observed output, uncontrolled noise, no scenario system, no quality metrics, no determinism guarantees.

B.2 SOG Version 2

V2 is a completely rebuilt, modular, deterministic benchmarking platform.

Table B.1: V1 vs V2

Dimension	V1	V2
Architecture	Monolithic script	9 core modules
Behavioral modeling	None	Latent propensity + SHA256 jitter
Household dynamics	Isolated records	Full household lifecycle
Life events	None	MOVE, COHABIT, BIRTH, DIVORCE, LEAVE_HOME
Truth layer	None	Normalized truth tables
Observed layer	Single dataset	Single, pairwise, N-way
Noise control	Random, unbounded	12 controllable noise types
Scenarios	One-off runs	11 canonical + regression
Quality metrics	None	Truth + scenario + ER benchmark
Determinism	Random	SHA256 + seeded RNG
Validation	Basic	Full output contract
Frontend	None	Streamlit UI
Testing	Minimal	322 tests

B.3 The Gap

V2 is not an incremental improvement. It is a fundamentally different system sharing only the core purpose. The architectural leap from random row generation to deterministic longitudinal simulation with household dynamics, truth/observed separation, and scenario-based benchmarking is substantial. The complete rebuild was necessary because V1's architecture could not support the features that make SOG a benchmarking platform.

APPENDIX C

Glossary

Table C.1: Term Definitions

Term	Definition
Baseline	Phase-1 synthetic population CSV that Phase 2 selects from
Cardinality	Matching relationship: one_to_one, one_to_many, many_to_one, many_to_many
Crosswalk	Table mapping records across datasets to the same entity
Determinism	Same inputs always produce identical outputs
Emission	Generating observed datasets from truth with noise/duplication
Entity	Real-world person represented by a PersonKey
Entity Record Map	Canonical bridge between PersonKeys and emitted records
Event	Life event: MOVE, COHABIT, BIRTH, DIVORCE, LEAVE_HOME
Household	Group sharing a residence, tracked with type and interval
Jitter	Per-entity random variation from SHA256 hashing
Latent Propensity	Hidden behavioral score (mobility, partnership, fertility)
Manifest	JSON describing all files in a run and their schemas
Noise	Controlled corruption: typos, masks, swaps, etc.
Observed Layer	Noisy denormalized dataset files an ER system ingests
Overlap	Fraction of entities appearing in both/all datasets
Quality Report	JSON measuring truth consistency, scenario, benchmark
Redundancy	Multiple Phase-1 rows for the same person
Scenario	YAML defining simulation params, emission, noise targets
Selection	Filtering/sampling Phase-1 to create a scenario cohort
SHA256 Jitter	Deterministic variation from SHA256(seed, salt, PersonKey)
Simulation	Time-stepped state machine applying life events
Snapshot	Point-in-time truth view for building observed datasets
Topology	Dataset structure: single, pairwise, or N-way
Truth Layer	Normalized, consistent tables representing the hidden world
Validator	Output contract checker verifying completeness